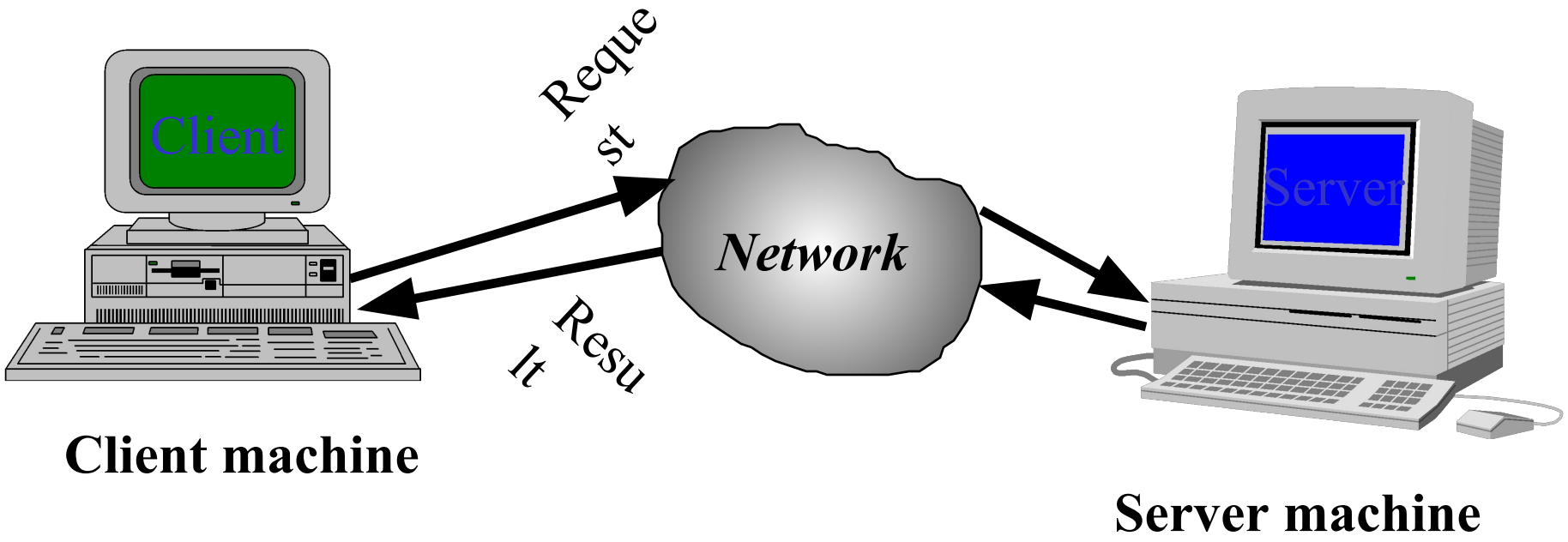
Network Programming and Java Sockets

Elements of C-S Computing

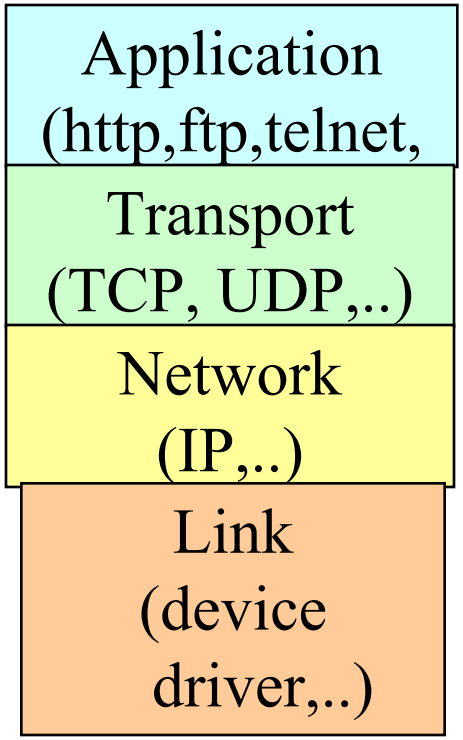
a client, a server, and network



Networking Basics

- **Applications Layer**
  - **Standard apps**
    - **HTTP**
    - **FTP**
    - **Telnet**
  - **User apps**
- **Transport Layer**
  - **TCP**
  - **UDP**
  - **Programming Interface:**
    - **Sockets**
- **Network Layer**
  - **IP**
- **Link Layer**
  - **Device drivers**

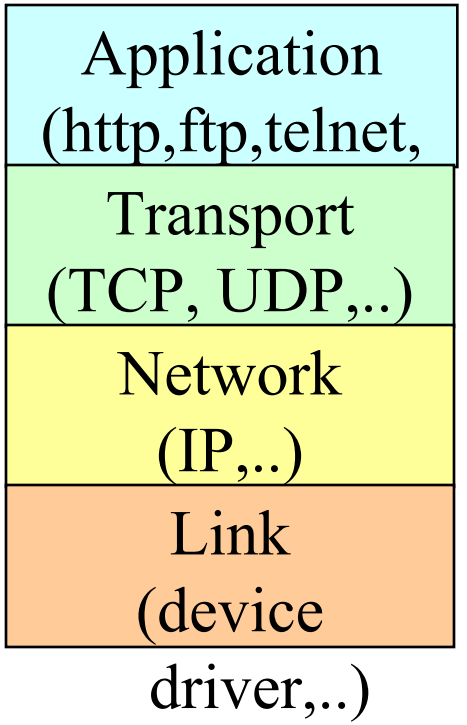
- TCP/IP Stack



Networking Basics

- **TCP (Transport Control Protocol) is a connection-oriented protocol that provides a reliable flow of data between two computers.**
- **Example applications:**
  - **HTTP**
  - **FTP**
  - **Telnet**

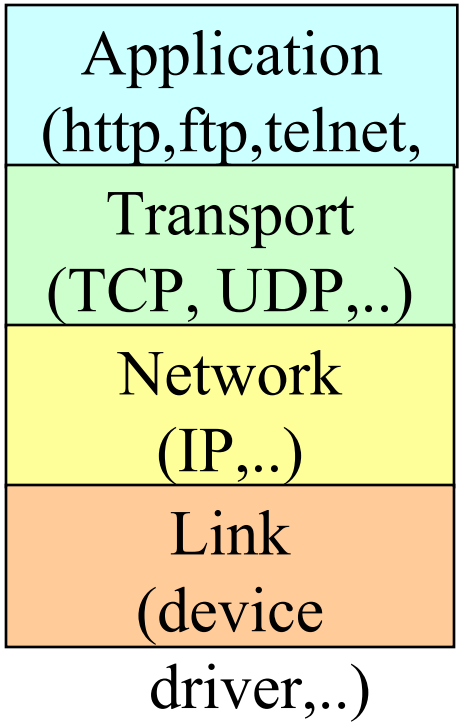
- **TCP/IP Stack**



Networking Basics

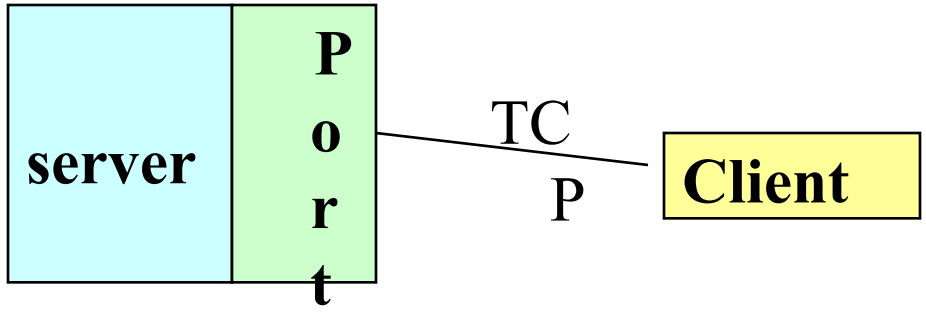
- **UDP (User Datagram Protocol) is a protocol that sends independent packets of data, called *datagrams*, from one computer to another with no guarantees about arrival.**
- **Example applications:**
  - **Clock server**
  - **Ping**

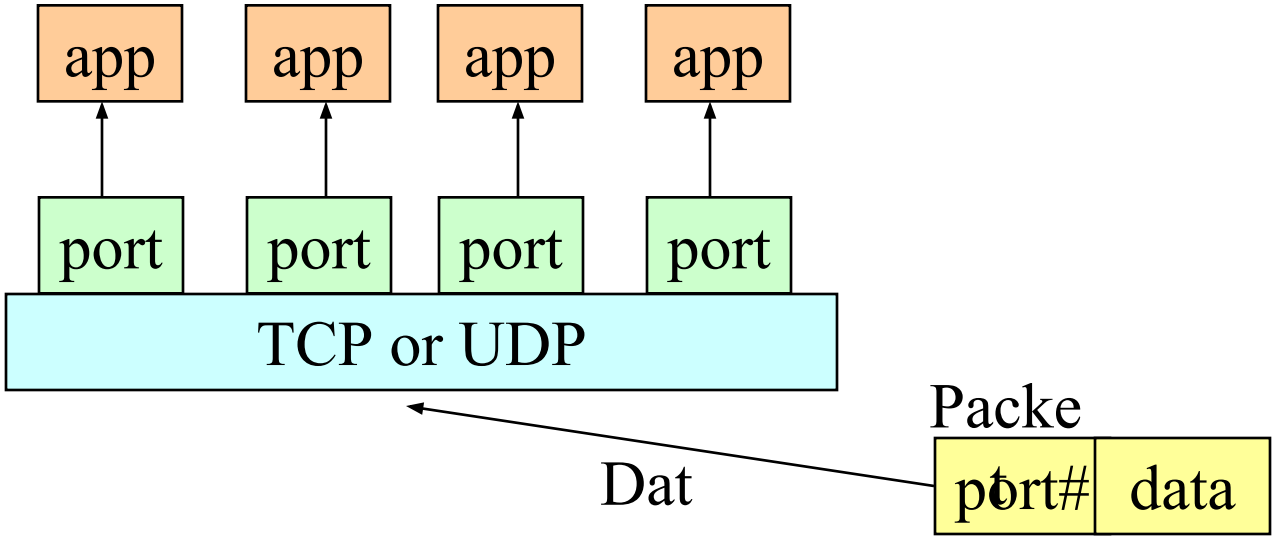
- TCP/IP Stack



Understanding Ports

- The TCP and UDP protocols use *ports* to map incoming data to a particular *process* running on a computer.





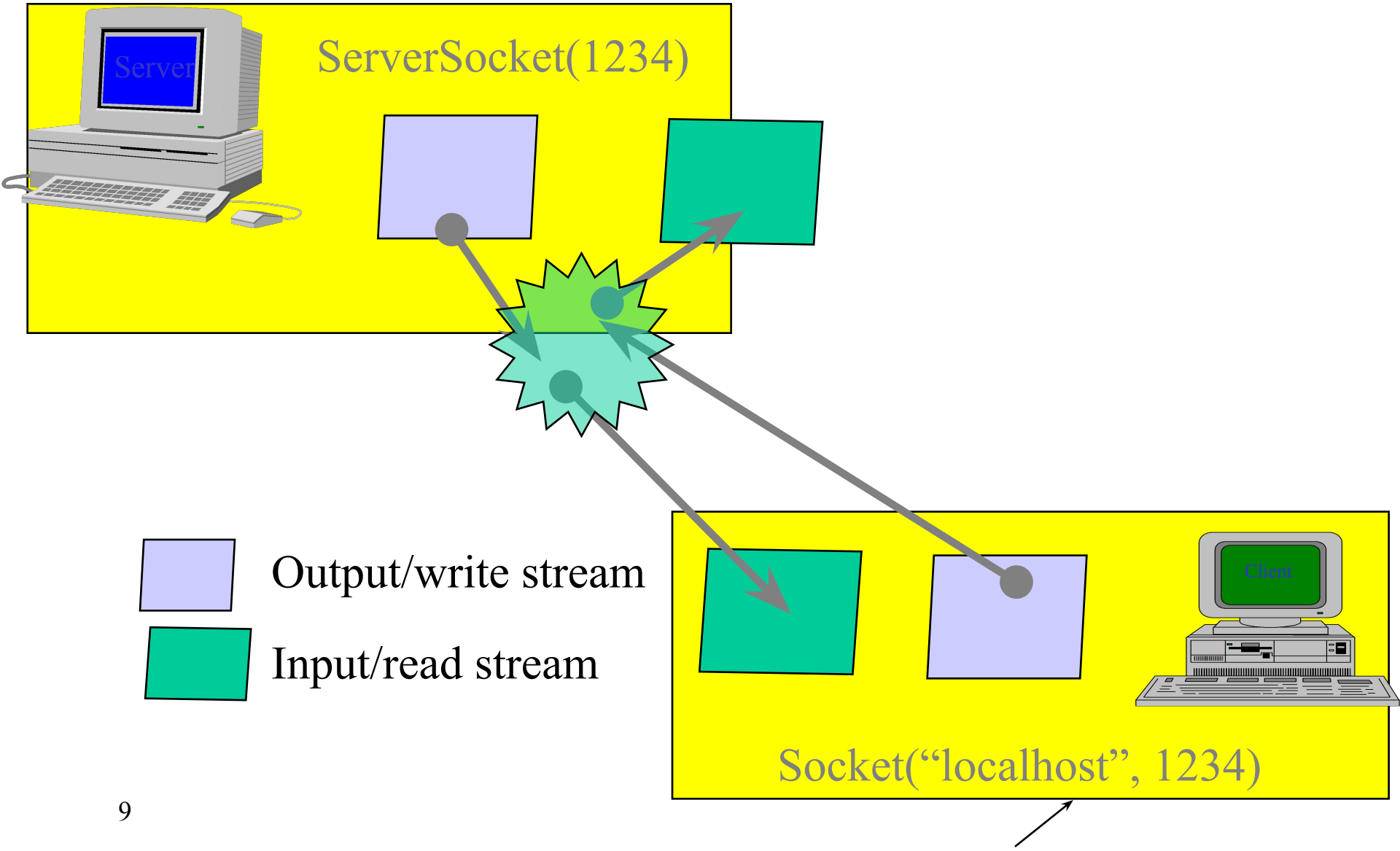
Understanding Ports

- Port is represented by a positive (16-bit) integer value
- Some ports have been reserved to support common/well known services:
  - ftp 21/tcp
  - telnet 23/tcp
  - smtp 25/tcp
  - login 513/tcp
- User level process/services generally use port number value >= 1024

Sockets and Java Socket Classes

- A socket is an endpoint of a two-way communication link between two programs running on the network.
- A socket is bound to a port number so that the TCP layer can identify the application that data destined to be sent.
- Java's .net package provides two classes:
  - **Socket – for implementing a client**
  - **ServerSocket – for implementing a server**

Java Sockets



Implementing a Server

1. Open the Server Socket:

```
ServerSocket server;
 DataOutputStream os;
 DataInputStream is;
 server = new ServerSocket( PORT );
```

2. Wait for the Client Request:

```
Socket client = server.accept();
```

3. Create I/O streams for communicating to the client

```
is=new DataInputStream(client.getInputStream());
os=new DataOutputStream(client.getOutputStream());
```

4. Perform communication with client

```
Receive from client:
String line = is.readLine();
Send to client:
os.writeBytes("Hello\n");
```

5. Close sockets: `client.close();`

Implementing a Client

1. Create a Socket Object:

```
client = new Socket( server, port_id );
```

2. Create I/O streams for communicating with the server.

```
is = new DataInputStream(client.getInputStream() );
os = new DataOutputStream( client.getOutputStream() );
```

3. Perform I/O or communication with the server:

   - Receive data from the server:

```
String line = is.readLine();
```

   - Send data to the server:

```
os.writeBytes("Hello\n");
```

4. Close the socket when done:

```
client.close();
```

Creating a server that sends data

```
import java.net.*;
import java.io.*;
public class SimpleServer {
  public static void main(String args[]) throws IOException {
    // Register service on port 1234
    ServerSocket s = new ServerSocket(1234);
    Socket s1=s.accept(); // Wait and accept a connection
    // Get a communication stream associated with the socket
    OutputStream s1out = s1.getOutputStream();
    DataOutputStream dos = new DataOutputStream (s1out);
    // Send a string!
    dos.writeUTF("Hi there");
    // Close the connection, but not the server socket
    dos.close();
    s1out.close();
    s1.close();
  }
}
```

Creating a client that receives data

```
import java.net.*;
import java.io.*;
public class SimpleClient {
  public static void main(String args[]) throws IOException
{    // Open your connection to a server, at port 1234
    Socket s1 = new Socket("localhost",1234);
// Get an input file handle from the socket and read the input
    InputStream s1In = s1.getInputStream();
    DataInputStream dis = new DataInputStream(s1In);
    String st = new String (dis.readUTF());
    System.out.println(st);
    // When done, just close the connection and exit
    dis.close();
    s1In.close();
    s1.close();
  }
}
```

Run

- Run Server
  - java SimpleServer
- Run Client on any machine
  - java SimpleClient

  Hi there

Server in Loop: Always up

```
// SimpleServerLoop.java: a simple server program that runs forever in a single thead
import java.net.*;
import java.io.*;
public class SimpleServerLoop {
  public static void main(String args[]) throws IOException {
    // Register service on port 1234
    ServerSocket s = new ServerSocket(1234);
    while(true)
    {
          Socket s1=s.accept(); // Wait and accept a connection
          // Get a communication stream associated with the socket
          OutputStream s1out = s1.getOutputStream();
          DataOutputStream dos = new DataOutputStream (s1out);
          // Send a string!
          dos.writeUTF("Hi there");
          // Close the connection, but not the server socket
          dos.close();
          s1out.close();
          s1.close();
    }
  }
}
```

Java API for UDP Programming

- Java API provides datagram communication by means of two classes
  - DatagramPacket

    | Msg | length | Host | serverPort |

  - DatagramSocket

UDP Client: Sends a Message and Gets reply

```
import java.net.*;
import java.io.*;
public class UDPClient
{
   public static void main(String args[]){
      // args give message contents and server hostname
      DatagramSocket aSocket = null;
        try {
            aSocket = new DatagramSocket();
            byte [] m = args[0].getBytes();
            InetAddress aHost = InetAddress.getByName(args[1]);
            int serverPort = 6789;
            DatagramPacket request = new DatagramPacket(m,  args[0].length(), aHost, serverPort);
            aSocket.send(request);
            byte[] buffer = new byte[1000];
            DatagramPacket reply = new DatagramPacket(buffer, buffer.length);
            aSocket.receive(reply);
            System.out.println("Reply: " + new String(reply.getData()));
         }
         catch (SocketException e){System.out.println("Socket: " + e.getMessage());}
         catch (IOException e){System.out.println("IO: " + e.getMessage());}
         finally
        {
           if(aSocket != null) aSocket.close();
        }
     }
}
```

UDP Sever: repeatedly received a request and sends it back to the client

```
import java.net.*;
import java.io.*;
public class UDPServer{
        public static void main(String args[]){
        DatagramSocket aSocket = null;
           try{
              aSocket = new DatagramSocket(6789);
              byte[] buffer = new byte[1000];
              while(true){
                  DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                aSocket.receive(request);
                DatagramPacket reply = new DatagramPacket(request.getData(),
                     request.getLength(), request.getAddress(), request.getPort());
                aSocket.send(reply);
              }
           }catch (SocketException e){System.out.println("Socket: " + e.getMessage());}
            catch (IOException e) {System.out.println("IO: " + e.getMessage());}
        finally {if(aSocket != null) aSocket.close();}
    }
}
```

Conclusion

- Programming client/server applications in Java is fun and challenging.
- Programming socket programming in Java is much easier than doing it in other languages such as C.
- Keywords:
  - Clients, servers, TCP/IP, port number, sockets, Java sockets